Discovery Executive Summary

Project: Discovery-PingCRM-03August · **Generated:** 03/08/2026, 16:07:03

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by legacy React component library (F5 High Risk), missing service layer (H2 Moderate), and direct DB access in controllers (H3 Moderate).

EXECUTIVE SUMMARY

PingCRM is a dual-layer CRM + IVR enterprise application built on Laravel and React with Inertia.js. The codebase exhibits a hybrid architecture with some clean patterns (slim RESTful routes, model scopes) but significant gaps in layering: business logic is scattered across controllers and utility classes, a dedicated Service Layer is absent, and the IVR and CRM domains are tightly coupled without anti-corruption boundaries. The most severe risk is the legacy widget library (180+ class-based React components) coexisting with modern functional components, creating maintenance and consistency challenges. Direct database access in controllers and a 261-line Reports component further violate separation of concerns and increase testing friction.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating

H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	152 LOC avg	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	5 direct accesses	MODERATE
H3	Missing Repository Pattern	Direct DB/ORM access points	<10	10–20	>20	8+ points	MODERATE
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 observed	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (IvrAccountContext)	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	85%	MODERATE
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 observed	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	2 observed	MODERATE
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	~10%	MODERATE
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	176 LOC avg	MODERATE
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	0 (Inertia.js SSR)	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (Reports at 261)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 (Inertia)	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	180+ class components	HIGH RISK

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1	Extract validation into Form Request classes; move report calculations into ReportService	MODERATE	HIGH
H2	Create ContactService, ReportService with reusable query/filter methods; controllers become thin adapters	MODERATE	HIGH
H3	Create Eloquent models for IVR tables; centralize all DB::table() calls into repositories	MODERATE	HIGH
H6	Replace raw DB::table() in ReportsController with named model scopes or repository methods	MODERATE	HIGH
H8	Introduce bounded contexts (app/Domain/Crm, app/Domain/Ivr); define anti-corruption layer	MODERATE	MEDIUM

H9	Segregate organizations into domain-owned logic or use domain column with strict filtering	MODERATE	MEDIUM
F1	Extract filter state into custom hooks (useReportFilters); move utilities to separate modules	MODERATE	MEDIUM
F5	Audit and remove unused legacy widgets; migrate top 20 to function components; add ESLint ban on class components	HIGH RISK	CRITICAL

1.5 Expected Outcomes

- **Testability:** Services unit-testable independently; repositories hide schema; components presentation-only.
- **Reusability:** Report logic and queries invokable from CLI, jobs, and APIs without duplication.
- **Maintainability:** Clear domain folder structure; anti-corruption layers prevent accidental coupling.
- **Scalability:** Domains evolve independently; multi-tenancy centralized in repositories.
- **Developer Experience:** Consistent modern component patterns; clear separation of concerns.
- **Resilience:** Legacy widget removal; modern error boundaries prevent single-component crashes.
